CLOUDGUARD

Final Product Requirements Document

Capital One Cloud Breach Analysis & Prevention • Version 1.0 • 27 October 2026

Status note: This is the proposed scope baseline based on the provided Antigravity audit. Audit-reported test results are identified as reported evidence, not an independent retest.

1. Purpose and product goal

CloudGuard is a local educational web application for a Computer Networks & Security (CNS) case study of the 2019 Capital One cloud data breach. It models selected failure modes—SSRF exposure, weak metadata access protections, excessive permissions and storage access-control gaps—and demonstrates how security controls change simulated access decisions.

Goal: show a repeatable before-and-after flow: select a vulnerable configuration, run a fixed local mock request, apply controls, repeat the request, and inspect the decision and audit event.

2. Scope boundaries

IN SCOPE	OUT OF SCOPE
Local Python standard-library server and static web UI	AWS deployment or real AWS accounts
Vulnerable/secured scenarios and mock SSRF, metadata, IAM and S3	Live exploitation, credential theft or external targets
Synthetic records, policy comparison and audit-event display	Production-grade AWS emulation, real shell or full SIEM
Concise incident education and repeatable demo	Claiming an exact reproduction of Capital One production

3. User flow

- Select vulnerable or secured baseline.
- Review control status and policy comparison.
- Run the fixed local mock scenario and observe each stage.
- Inspect audit events.
- Switch to secured configuration and repeat the same request.
- Compare outcomes and explain which control changed the decision.

4. Functional requirements

ID	Requirement
FR-01	Apply vulnerable/secured baselines; keep backend and UI state synchronized.
FR-02	Mock proxy blocks prohibited link-local/private targets in secured mode.
FR-03	Model IMDSv1/IMDSv2 locally; never contact real metadata endpoints.
FR-04	Evaluate synthetic credentials and permissions before simulated S3 actions.
FR-05	Use synthetic records and enforce configured identity/bucket checks.
FR-06	Record simulated access decisions and configuration changes.
FR-07/08	Compare policies and explain the historical case-study connection.
FR-09	Reset both UI and backend to the documented baseline.

5. Technical baseline

Layer	Current reported implementation
Backend	Python 3.11 standard library; http.server/socketserver; port 5000
Frontend	HTML, CSS and vanilla JavaScript under static/
Data/state	JSON scenario data; in-memory scenario state and audit events
Tests	test_simulation.py; audit reports 12 passed, 0 failed, exit code 0
Deployment	Local laptop; no real AWS integration

6. Current implementation status

Based on Antigravity's audit dated 27 October 2026. These statuses reflect the supplied report; they are not an independent retest.

Area	Audit status	Notes / remaining work
Server/static assets	Verified	Local server and static assets returned HTTP 200.
Scenario switcher	Verified	Backend state changes between vulnerable and secured.
SSRF / metadata	Partial	SSRF differential tested; add explicit IMDSv2 token handshake test.
IAM / S3	Partial	Outcomes differ; isolate-test bucket policy independently.
Breach pipeline	Tested	12-test suite reports vulnerable flow and secured blocking.
Frontend/API	Partial	Python suite does not verify all browser interactions.
Audit logs	Partial	In-memory only; lost after restart.
Reset button	Defect	UI reset does not reset backend configuration.
Terminal JS	Defect	Accidental global assignment around terminal.js line 80.
Topology	Illustrative	Client-side animation, not real network telemetry.

7. Remaining work, prioritized

Priority	Task	Acceptance evidence
P0	Fix Reset so backend and UI return to the same baseline.	Switch to secured, reset, then confirm /api/status and UI match.
P0	Remove accidental global assignment in terminal.js.	Code review; terminal simulator still works.
P1	Test IMDSv2 token handshake.	Test token issuance, valid token GET, missing/invalid token rejection.
P1	Test bucket-policy enforcement independently.	Broad IAM plus restricted bucket policy still denies the target access.
P1	Perform browser-level integration checks.	Manual checklist or browser automation for buttons, tabs and flow.
P2	Decide whether persistent logs are necessary.	Document in-memory limitation or add SQLite if time permits.
P2	Finalize README and demo script.	Setup, tests, limitations and demo steps documented.

8. Final acceptance criteria

- Same unauthorized mock request gives different results in vulnerable and secured scenarios.
- Secured mode denies unauthorized access while allowing an authorized mock request.
- Backend policy logic determines access; UI text alone does not.
- Reset synchronizes frontend and backend state.
- Audit events identify simulated actions and decisions.
- Tests cover main flow, IMDSv2 token behavior and isolated bucket-policy enforcement.
- All data, credentials, services and attack actions are clearly labeled as simulated.
- The presentation distinguishes historical facts from the simplified local model.

9. Change control

This PRD is the project scope baseline. Any feature or architecture change should record the reason, owner, impact and acceptance test before implementation. Work one milestone at a time and review evidence before proceeding. Avoid scope expansion.

10. Factual framing

CloudGuard is an educational local simulation of selected security concepts associated with the Capital One breach. It is not Capital One's production configuration, does not contact real AWS services, and its visual topology/terminal are illustrative emulators.